CPU Scheduling – المفاهيم الاساسية لجدولة وحدة المعالجة المركزية

💡 الجزء الأول: لماذا نحتاج جدولة المعالج؟

🔑 المشكلة الأساسية

السيناريو:

```text
معالج واحد (أو عدد محدود من النوى)
مئات البرامج تريد العمل في نفس الوقت
كيف ندير هذا؟ 🤔
```

الواقع القاسي:

- في نظام **Single-core**: عملية **واحدة فقط** تعمل في لحظة معينة
- الباقي **ينتظر** دوره

🎯 الحل: Multiprogramming

التعريف:

- **تعدد البرامج:** إبقاء عدة عمليات في الذاكرة
- **التبديل السريع:** عندما واحدة تنتظر، نشغل أخرى
- **ببساطة:** "لا نضيع ولا ثانية من وقت المعالج"

الآلية:

```text
عملية A تحسب → تتوقف لانتظار I/O
↓
نظام التشغيل يسحب المعالج منها
↓
يعطيه لعملية B
↓
B تشتغل بينما A تنتظر ✅
```

⭐ **للحفظ:** Multiprogramming = **عدة عمليات في الذاكرة + تبديل سريع**

🎯 الهدف الأساسي

الهدف الرئيسي لـ CPU Scheduling:

- **تعظيم استخدام المعالج** (Maximize CPU Utilization)
- **المعالج مشغول دائماً** - لا وقت ضائع
- **ببساطة:** "المعالج ما يرتاح!"

القاعدة الذهبية:

```text
المعالج فاضي (Idle) = ❌ هدر موارد
المعالج مشغول دائماً = ✅ كفاءة عالية
```

⭐ **للامتحان:**

- السؤال: "ما الهدف من CPU Scheduling؟"
- الجواب: **Maximize CPU utilization**

🔄 الجزء الثاني: دورة CPU-I/O Burst

🔑 المفهوم الأساسي

الملاحظة الذهبية:

- أي برنامج **لا يحسب طول الوقت**
- بل يتناوب بين **حسابات** و**انتظار**

التقسيم:

1. CPU Burst (نوبة المعالج):

- العملية تنفذ **تعليمات** على المعالج
- **حسابات فعلية** (1+1، مقارنات، إلخ)
- **ببساطة:** "الشغل الفعلي"

2. I/O Burst (نوبة الإدخال/الإخراج):

- العملية **تنتظر**
- انتظار **قراءة ملف** أو **كتابة** أو **إدخال مستخدم**
- **ببساطة:** "وقت الانتظار"

🔍 التشبيه: الشيف والفرن

المثال:

```text
الشيف يفرم البصل → CPU Burst (شغل فعلي)
↓
يضع الطبخة في الفرن → بداية I/O Burst
↓
ينتظر الطبخة تستوي → I/O Burst (انتظار)
↓
يخرج الطبخة → نهاية I/O Burst
↓
يزين الطبق → CPU Burst (شغل فعلي مرة أخرى)
```

الذكاء:

- بدلاً من **الوقوف فاضياً** أمام الفرن
- الشيف **يفرم بصل** لطبخة أخرى
- **هذا هو Multiprogramming!**

🔄 الدورة الكاملة

Process Execution Cycle:

```text
بداية العملية
↓
CPU Burst (حسابات)
↓
I/O Burst (انتظار I/O)
↓
CPU Burst (حسابات)
↓
I/O Burst (انتظار)
↓
... (تكرار)
↓
CPU Burst الأخير (طلب إنهاء)
↓
انتهاء العملية
```

⚠️ **ملاحظة مهمة:**

- العملية **تبدأ** بـ CPU Burst
- العملية **تنتهي** بـ CPU Burst (يحتوي طلب الإنهاء)

⭐ **للحفظ:** العملية = **تناوب** بين CPU Burst و I/O Burst

📊 خصائص CPU Bursts

الملاحظة الإحصائية المهمة:

أغلب البرامج:

- عدد **كبير** من CPU Bursts **القصيرة**
- عدد **قليل** من CPU Bursts **الطويلة**

التوزيع:

```text
قصيرة: ████████████████ (كثيرة)
متوسطة: ████ (قليلة)
طويلة: ▌ (نادرة جداً)
```

التصنيف:

1. **CPU-Bound Process (عملية محسوبات):**
   - CPU Bursts **طويلة**
   - I/O Bursts **قليلة**
   - **مثال:** حسابات رياضية معقدة
2. **I/O-Bound Process (عملية إدخال/إخراج):**
   - CPU Bursts **قصيرة وكثيرة**
   - I/O Bursts **كثيرة**
   - **مثال:** محرر نصوص (ينتظر المستخدم يكتب)

⭐ **للامتحان:**

- السؤال: "أغلب البرامج تحتوي؟"
- الجواب: **عدد كبير من CPU Bursts القصيرة**

📋 الجزء الثالث: مجدول المعالج (CPU Scheduler)

🔑 التعريف والدور

CPU Scheduler:

- **جزء من نظام التشغيل**
- **يختار** عملية من Ready Queue
- **يعطيها** المعالج للتنفيذ

الوظيفة:

```text
المعالج فاضي
↓
Scheduler يفحص Ready Queue
↓
يختار عملية حسب السياسة المتبعة
↓
يعطيها المعالج
```

⭐ **للحفظ:** CPU Scheduler = **يختار من Ready Queue**

📊 Ready Queue (طابور الاستعداد)

التعريف:

- **طابور** العمليات الجاهزة للتنفيذ
- موجودة في **الذاكرة الرئيسية**
- **تنتظر** دورها على المعالج

⚠️ **ملاحظة مهمة:**

- Ready Queue **ليس بالضرورة** FIFO (First-In-First-Out)
- قد يكون:
  - **Priority Queue** (طابور أولويات)
  - **Tree** (شجرة)
  - **Unordered List** (قائمة غير مرتبة)
- **نوع الهيكل** يعتمد على **خوارزمية الجدولة**

⭐ **للحفظ:** Ready Queue **ليس دائماً FIFO** - يعتمد على الخوارزمية

🎯 متى يعمل Scheduler؟

Scheduler ينشط عندما:

1. عملية تنتهي
2. عملية تنتقل لـ Waiting (I/O)
3. Interrupt يحدث
4. عملية جديدة تصبح Ready

⚔️ الجزء الرابع: Preemptive vs Nonpreemptive

🔑 أهمية الموضوع

⚠️ هذا أهم موضوع في الجابتر للامتحانات!

السؤال المحوري:

```text
عملية تعمل على المعالج
هل يمكن سحب المعالج منها قسراً؟
```

📊 الحالات الأربعة لقرارات الجدولة

قرارات CPU Scheduling تحدث في 4 حالات:

الحالة 1:

- العملية من **Running → Waiting**
- **مثال:** طلبت I/O operation

الحالة 2:

- العملية من **Running → Ready**
- **مثال:** حدث Interrupt (انتهى time slice)

الحالة 3:

- العملية من **Waiting → Ready**
- **مثال:** I/O اكتملت

الحالة 4:

- العملية **تنتهي** (Terminates)

🤝 Nonpreemptive Scheduling (غير قسري)

التعريف:

- العملية **لا تُجبر** على ترك المعالج
- تتركه **فقط** عندما:
  - **تنتهي** (Terminates) ✅
  - **تطلب I/O** (تنتقل لـ Waiting) ✅

الحالات:

- **فقط الحالة 1 و 4**

الصفة:

- **"النظام المؤدب"**
- العملية **تتحكم** في المعالج
- النظام **ينتظر** حتى تنتهي أو تطلب I/O

المثال:

```text
عملية A تعمل
↓
حتى لو جاءت عملية B أهم منها
↓
A تستمر حتى تنتهي أو تطلب I/O
↓
النظام لا يقاطعها ❌
```

المميزات:

- ✅ **بسيط** جداً
- ✅ **أقل Context Switches**
- ✅ **أقل Overhead**

العيوب:

- ❌ **استجابة بطيئة**
- ❌ عملية واحدة قد **تحتكر** المعالج
- ❌ **Starvation** للعمليات المهمة

⭐ **للحفظ:** Nonpreemptive = العملية **تترك المعالج طوعاً** (حالة 1 و 4)

👊 Preemptive Scheduling (قسري)

التعريف:

- نظام التشغيل **يمكنه سحب المعالج** من العملية
- **حتى لو** العملية لم تنته ولم تطلب I/O

الحالات:

- **الحالة 2 و 3** أيضاً

الصفة:

- **"النظام المسيطر"**
- نظام التشغيل **له الكلمة العليا**
- **يقاطع** العملية عند الحاجة

متى يحدث؟

1. Time Quantum انتهى:

- العملية أخذت وقتها المحدد
- النظام **يسحب** المعالج منها
- **ينقلها** من Running → Ready

2. عملية أعلى أولوية أصبحت Ready:

- عملية مهمة جداً جاءت
- النظام **يقاطع** العملية الحالية
- **يعطي** المعالج للأهم

المثال:

```text
عملية A (أولوية منخفضة) تعمل
↓
عملية B (أولوية عالية جداً) أصبحت Ready
↓
النظام يقاطع A فوراً
↓
يعطي المعالج لـ B ✅
```

المميزات:

- ✅ **استجابة سريعة**
- ✅ **عدالة** أفضل
- ✅ **لا احتكار** للمعالج
- ✅ مناسب للأنظمة **Real-time**

العيوب:

- ❌ **أكثر Context Switches**
- ❌ **أكثر Overhead**
- ❌ **أكثر تعقيداً**
- ❌ مشاكل **Race Conditions** محتملة

⭐ **للحفظ:** Preemptive = النظام **يسحب المعالج قسراً** (حالة 2 و 3 أيضاً)

📊 جدول المقارنة الشامل

الميزة	Nonpreemptive	Preemptive
سحب المعالج	❌ لا	✅ نعم قسراً
متى تترك CPU	طوعاً فقط	أي وقت
الحالات	1 و 4	1 و 2 و 3 و 4
Context Switches	أقل	أكثر
Overhead	أقل	أكثر
الاستجابة	بطيئة	سريعة
التعقيد	بسيط	معقد
الاحتكار	ممكن	ممنوع
الاستخدام الحديث	نادر	شائع جداً

🌍 الأنظمة الحديثة

كل الأنظمة الحديثة تستخدم Preemptive:

- Windows ✅
- Linux ✅
- macOS ✅
- Android ✅
- iOS ✅

لماذا؟

- **استجابة أفضل** للمستخدم
- **عدالة** في توزيع المعالج
- **لا احتكار** من برنامج واحد

⭐ **للامتحان:**

- السؤال: "الأنظمة الحديثة تستخدم أي نوع؟"
- الجواب: **Preemptive Scheduling**

🏃 الجزء الخامس: الموزع (Dispatcher)

🔑 التعريف

Dispatcher:

- **وحدة** في نظام التشغيل
- **تنفذ** قرار Scheduler
- **تعطي** التحكم فعلياً للعملية المختارة
- **ببساطة:** "رجل التنفيذ - العضلات"

الفرق الحرج:

```text
Scheduler (المجدول):
  - العقل المفكر 🧠
  - يقرر "من" يعمل

Dispatcher (الموزع):
  - العضلات المنفذة 💪
  - ينفذ القرار فعلياً
```

⭐ **للحفظ:** Scheduler **يقرر** - Dispatcher **ينفذ**

🔧 مهام Dispatcher الثلاثة

الموزع يقوم بـ 3 مهام أساسية:

1️⃣ Context Switching (تبديل السياق):

ماذا يحدث؟

- **حفظ** حالة العملية القديمة (السجلات، Program Counter، إلخ)
- **تحميل** حالة العملية الجديدة

الهدف:

- العملية القديمة تستطيع **العودة لاحقاً** من نفس النقطة
- العملية الجديدة تبدأ من **آخر نقطة** وصلت لها

2️⃣ Switching to User Mode (التحول لوضع المستخدم):

السبب:

- نظام التشغيل يعمل في **Kernel Mode**
- العمليات العادية تعمل في **User Mode**

الآلية:

- Dispatcher يغير **وضع** المعالج
- من Kernel Mode → User Mode

3️⃣ Jumping to Proper Location (القفز للموقع الصحيح):

الهدف:

- توجيه **Program Counter** للمكان الصحيح
- العملية تبدأ التنفيذ من **السطر الصحيح**

⭐ **للحفظ:** Dispatcher = **Context Switch** + **User Mode** + **Jump**

⏱️ Dispatch Latency (زمن تأخير التوزيع)

🔑 التعريف

Dispatch Latency:

- **الوقت** الذي يأخذه Dispatcher
- لـ **إيقاف** عملية و**بدء** عملية أخرى
- **ببساطة:** "وقت التبديل"

الصيغة:

```text
Dispatch Latency = الوقت من (قرار التبديل) حتى (بداية تنفيذ العملية الجديدة)
```

📊 أهمية Dispatch Latency

لماذا مهم؟

1. يحدث بكثرة:

- الحاسوب يبدل بين العمليات **آلاف المرات** في الثانية
- كل Context Switch = Dispatch Latency

2. Pure Overhead:

- **لا عمل مفيد** يتم خلاله
- **مجرد** نقل السيطرة
- **وقت ضائع** تماماً

المثال:

```text
إذا:
- Context Switch = 1000 مرة في الثانية
- Dispatch Latency = 1 ms

النتيجة:
- ثانية كاملة ضائعة = %100 overhead! ❌

إذا:
- Dispatch Latency = 0.01 ms

النتيجة:
- 10 ms فقط ضائعة = %1 overhead ✅
```

⭐ **المطلوب:**

- Dispatch Latency يجب أن يكون **صغير جداً**
- **أسرع ما يمكن**
- النظام الجيد: **أقل من 1 microsecond**

⭐ **للامتحان:**

- السؤال: "Dispatch Latency يجب أن يكون؟"
- الجواب: **صغير جداً - لأنه pure overhead**

📊 الجزء السادس: الصورة الكاملة

🔄 عملية الجدولة الكاملة

```text
1. عملية A تعمل على المعالج
   ↓
2. يحدث أحد الأحداث (I/O، Interrupt، Termination)
   ↓
3. CPU Scheduler ينشط
   ↓
4. يفحص Ready Queue
   ↓
5. يختار عملية B (حسب السياسة)
   ↓
6. Dispatcher ينشط
   ↓
7. Context Switch: يحفظ A، يحمل B
   ↓
8. يحول لـ User Mode
   ↓
9. يقفز لموقع B
   ↓
10. B تبدأ التنفيذ ✅
```

الوقت الضائع: فقط خطوة 7 (Dispatch Latency)

📊 العلاقة بين المفاهيم

```text
Multiprogramming
↓ (يتطلب)
CPU Scheduling
↓ (يستخدم)
CPU Scheduler (يقرر)
↓
Dispatcher (ينفذ)
↓
Context Switch
↓
العملية الجديدة تعمل
```

⚠️ الأخطاء الشائعة في الامتحانات

❌ خطأ 1: Scheduler = Dispatcher

الصحيح: مختلفان - Scheduler يقرر، Dispatcher ينفذ

❌ خطأ 2: Ready Queue دائماً FIFO

الصحيح: ليس دائماً - يعتمد على الخوارزمية

❌ خطأ 3: Nonpreemptive أفضل دائماً

الصحيح: Preemptive أفضل للأنظمة الحديثة

❌ خطأ 4: CPU Burst = I/O Burst

الصحيح: مختلفان - CPU للحسابات، I/O للانتظار

❌ خطأ 5: Dispatch Latency غير مهم

الصحيح: مهم جداً - يحدث آلاف المرات

🎯 الفخاخ الشائعة

```text
❌ Scheduler = Dispatcher → ✅ مختلفان
❌ Ready Queue = FIFO → ✅ ليس دائماً
❌ Nonpreemptive أفضل → ✅ Preemptive للحديثة
❌ CPU Burst = I/O Burst → ✅ مختلفان تماماً
❌ Dispatch Latency غير مهم → ✅ مهم جداً
```

🔥 ورقة الغش النهائية

📝 المفاهيم الأساسية

```text
الهدف: Maximize CPU utilization

Multiprogramming:
- عدة عمليات في الذاكرة
- تبديل سريع
- لا وقت ضائع

CPU Burst: حسابات فعلية
I/O Burst: انتظار I/O

Process Execution:
- تبدأ بـ CPU Burst
- تتناوب: CPU → I/O → CPU → I/O
- تنتهي بـ CPU Burst

أغلب البرامج:
- CPU Bursts قصيرة وكثيرة ✅
- CPU Bursts طويلة قليلة
```

📝 Scheduler vs Dispatcher

```text
Scheduler (المجدول):
- العقل 🧠
- يقرر "من" يعمل
- يختار من Ready Queue

Dispatcher (الموزع):
- العضلات 💪
- ينفذ القرار
- 3 مهام:
  1. Context Switch
  2. User Mode
  3. Jump

Dispatch Latency:
- وقت التبديل
- pure overhead
- يجب أن يكون صغير جداً ✅
```

📝 Preemptive vs Nonpreemptive

```text
Nonpreemptive (غير قسري):
- الحالات: 1 و 4 فقط
- تترك طوعاً
- أقل Context Switches
- أبسط
- استجابة بطيئة
- قد تحتكر المعالج

Preemptive (قسري):
- الحالات: 1 و 2 و 3 و 4
- يُسحب قسراً
- أكثر Context Switches
- أعقد
- استجابة سريعة ✅
- لا احتكار ✅
- الأنظمة الحديثة ✅
```

⭐ نقاط حرجة للامتحان

```text
1. الهدف؟ Maximize CPU utilization ✅
2. Ready Queue دائماً FIFO؟ لا ❌
3. أغلب البرامج؟ CPU Bursts قصيرة وكثيرة ✅
4. Nonpreemptive حالات؟ 1 و 4 فقط ✅
5. Preemptive حالات؟ 1 و 2 و 3 و 4 ✅
6. الأنظمة الحديثة؟ Preemptive ✅
7. Scheduler يقوم بـ؟ يقرر ✅
8. Dispatcher يقوم بـ؟ ينفذ ✅
9. Dispatch Latency؟ صغير جداً ✅
10. Single-core؟ عملية واحدة فقط ✅
```

معايير وخوارزميات الجدولة - Scheduling Criteria & Algorithms

🎯 الصورة الكاملة

السؤال الأساسي: كيف نعرف أن خوارزمية أفضل من أخرى؟

الجواب: نقيسها بـ **5 معايير** - ثم نطبق **6 خوارزميات** مختلفة

📏 القسم الأول: معايير الجدولة (المسطرة)

المعايير الخمسة:

1. CPU Utilization (استغلال المعالج)

- **التعريف:** نسبة الوقت الذي يكون فيه المعالج مشغولاً
- **الهدف:** Maximize (نعظمه)
- **المدى:** 40% - 90%
- **ببساطة:** "المعالج مشغول = ✅ جيد"

---

2. Throughput (الإنتاجية)

- **التعريف:** عدد العمليات التي **تنتهي** في وحدة الزمن
- **الهدف:** Maximize
- **مثال:** 10 عمليات/ثانية = جيد، 100 عملية/ثانية = ممتاز
- **ببساطة:** "كم عملية خلصنا؟"

3. Turnaround Time (زمن الدوران)

- **التعريف:** من **وصول العملية** حتى **انتهائها بالكامل**
- **الصيغة:** `Turnaround = Completion Time - Arrival Time`
- **يشمل:** Waiting + Execution + I/O
- **الهدف:** Minimize (نقلله)
- **ببساطة:** "الوقت الكلي من البداية للنهاية"

---

4. Waiting Time (زمن الانتظار) ⭐

- **التعريف:** مجموع الوقت في **Ready Queue فقط**
- **لا يشمل:** Execution ولا I/O
- **الهدف:** Minimize
- ⚠️ **الأهم:** المجدول يؤثر عليه **مباشرة**
- **ببساطة:** "كم قعد ينتظر بدون شغل؟"

5. Response Time (زمن الاستجابة)

- **التعريف:** من **تقديم الطلب** حتى **أول استجابة**
- ⚠️ **مهم:** ليس حتى الانتهاء - فقط **أول رد**
- **الهدف:** Minimize
- **الأهمية:** حرج في **الأنظمة التفاعلية**
- **ببساطة:** "كم استغرق حتى يبدأ؟"

📊 جدول المقارنة السريع

المعيار	التعريف	الهدف	الأهمية
CPU Utilization	نسبة انشغال CPU	Maximize	النظام كفء
Throughput	عدد العمليات المكتملة	Maximize	الإنتاجية
Turnaround Time	وصول → انتهاء	Minimize	رضا المستخدم
⭐ Waiting Time	وقت Ready Queue	Minimize	**الأهم**
Response Time	طلب → أول رد	Minimize	تفاعلية

🔢 القسم الثاني: الخوارزميات الستة

1️⃣ FCFS (First-Come, First-Served)

المبدأ: "الأول بالطابور يخدم أولاً"

الخصائص:

- ✅ Nonpreemptive
- ✅ يستخدم FIFO Queue
- ✅ **الأبسط** على الإطلاق

التقييم:

- ❌ Waiting Time طويل جداً
- ❌ يعاني من Convoy Effect

⚠️ Convoy Effect (تأثير القافلة)

المشكلة:

```text
عملية طويلة CPU-bound (مثلاً 100 ثانية)
وراءها عمليات قصيرة I/O-bound (كل واحدة ثانية)

النتيجة:
- الكل ينتظر الطويلة تخلص ❌
- استغلال سيء للموارد ❌
```

التشبيه: سيارة بطيئة بالطريق السريع - **الكل عالق وراءها**

⭐ للامتحان:

- السؤال: "ما عيب FCFS الأساسي؟"
- الجواب: **Convoy Effect - عمليات قصيرة تنتظر طويلة**

2️⃣ SJF (Shortest Job First)

المبدأ: "الأقصر أولاً"

الخصائص:

- ✅ يختار العملية بأقصر **CPU Burst**
- ✅ نوعان:
  - **Nonpreemptive SJF:** لا مقاطعة
  - **Preemptive SJF (SRTF):** مقاطعة إذا جاءت أقصر

⭐ الأمثلية (Optimality)

الحقيقة الذهبية:

- SJF يعطي **أقل متوسط Waiting Time ممكن**
- **أفضل من أي خوارزمية أخرى** رياضياً
- **Optimal** بشكل مثبت

⭐ **للحفظ:** SJF = **Optimal** - أقل Waiting Time

❌ المشكلة الكبرى

التحدي: كيف نعرف طول CPU Burst القادم؟

الواقع:

- **مستحيل** معرفته مسبقاً في الأنظمة التفاعلية
- **ممكن** في Batch Systems (معروف مسبقاً)

الحل: التوقع (Prediction)

📐 التوقع بالمتوسط الأسي

الفكرة: نستخدم تاريخ العملية للتوقع

المعادلة:

```text
τ(n+1) = α × t(n) + (1-α) × τ(n)
```

حيث:

- `τ(n+1)` = التوقع للـ Burst القادم
- `t(n)` = الطول **الفعلي** للـ Burst السابق
- `τ(n)` = التوقع السابق
- `α` = وزن ($0 \le \alpha \le 1$)

التفسير:

- `α = 0`: نعتمد فقط على التاريخ البعيد
- `α = 1`: نعتمد فقط على آخر Burst
- `α = 0.5`: توازن بينهما

⭐ **للامتحان:** SJF يستخدم **Exponential Average** للتوقع

3️⃣ Priority Scheduling (جدولة الأولوية)

المبدأ: "الأعلى أولوية أولاً"

الخصائص:

- ✅ كل عملية لها **Priority Number**
- ✅ **الأقل رقماً = الأعلى أولوية** (أو العكس حسب النظام)
- ✅ يمكن أن يكون **Preemptive** أو **Nonpreemptive**

⚠️ **ملاحظة:** SJF هي حالة خاصة (الأولوية = طول Burst)

❌ المشكلة: **Starvation** (المجاعة)

التعريف:

- عملية **منخفضة الأولوية** قد **لا تعمل أبداً**
- **Indefinite Blocking** (حجب لا نهائي)

السيناريو:

```text
عملية أولوية منخفضة في Ready Queue
↓
باستمرار تأتي عمليات أعلى أولوية
↓
المنخفضة تنتظر... وتنتظر... للأبد ❌
```

✅ الحل: **Aging** (التقادم)

الفكرة:

- **كلما انتظرت أكثر → أولويتك ترتفع**
- زيادة تدريجية مع الوقت

المثال:

```text
البداية: Priority = 10 (منخفضة)
بعد دقيقة: Priority = 9
بعد دقيقتين: Priority = 8
...
في النهاية: تصل الأولوية = 0 (عالية جداً) ✅
```

⭐ **للامتحان:**

- السؤال: "كيف نحل مشكلة Starvation؟"
- الجواب: **Aging - رفع الأولوية تدريجياً**

4️⃣ Round Robin (RR) - الجدولة الدورية

المبدأ: "كل واحد ياخد دوره بالتساوي"

الخصائص:

- ✅ **Preemptive**
- ✅ كل عملية تأخذ **Time Quantum** (q)
- ✅ مصممة لـ **Time-Sharing Systems**

🔄 الآلية

```text
Ready Queue: [P1, P2, P3]
Time Quantum = 10 ms

P1 يعمل 10 ms → يرجع لآخر الطابور
P2 يعمل 10 ms → يرجع لآخر الطابور
P3 يعمل 10 ms → يرجع لآخر الطابور
P1 يعمل 10 ms مرة أخرى...
```

إذا العملية انتهت قبل q: تترك CPU مباشرة

إذا لم تنته: Timer Interrupt → تعود للطابور

📊 التقييم

المميزات:

- ✅ **Response Time ممتاز**
- ✅ **عدالة** - كل واحد ياخد نفس الوقت
- ✅ مثالي لـ **Interactive Systems**

العيوب:

- ❌ **Waiting Time أطول** من SJF
- ❌ **Context Switch كثير** إذا q صغير

⚙️ اختيار Time Quantum

القاعدة الذهبية:

- **80% من العمليات** يجب أن تنهي Burst في **دورة واحدة**

إذا q كبير جداً:

- يصبح مثل **FCFS** ❌
- **لا فائدة** من RR

إذا q صغير جداً:

- **Context Switching كثير جداً** ❌
- **Overhead عالي**

القيم الشائعة: 10-100 ms

⭐ **للامتحان:** RR مثالي لـ **Interactive Systems** - Response Time ممتاز

5️⃣ Multilevel Queue (طوابير متعددة المستويات)

المبدأ: "كل نوع له طابوره"

الفكرة:

- بدلاً من طابور واحد → **عدة طوابير منفصلة**
- كل طابور لنوع معين من العمليات

📊 التقسيم النموذجي

```text
Priority Level 0: System Processes (أعلى أولوية)
Priority Level 1: Interactive Processes
Priority Level 2: Batch Processes (أقل أولوية)
```

الخصائص:

- ✅ كل طابور **خوارزميته الخاصة**
  - مثلاً: Interactive → **RR**
  - Batch → **FCFS**
- ✅ **لا انتقال** بين الطوابير

🔧 الجدولة بين الطوابير

نوعان:

1. Fixed Priority:

- الطابور الأعلى **له الأولوية المطلقة**
- الأدنى **يعمل فقط** إذا الأعلى فارغ

2. Time Slice:

- كل طابور يأخذ **نسبة من وقت CPU**
- مثلاً: 80% للـ Interactive، 20% للـ Batch

⚠️ المشكلة

Starvation للطوابير الدنيا:

- إذا الطابور العالي دائماً مشغول
- الطوابير الدنيا **لا تعمل أبداً**

⭐ **للحفظ:** Multilevel Queue = **طوابير منفصلة** - لا انتقال

6️⃣ Multilevel Feedback Queue (MLFQ)

المبدأ: "الطوابير الذكية - تتعلم من السلوك"

الفرق الجوهري عن MLQ:

- ✅ **العمليات تنتقل** بين الطوابير
- ✅ **ديناميكية** حسب السلوك
- ✅ **الأكثر مرونة وتعقيداً**

🔄 الآلية

القاعدة 1: العملية الجديدة:

- تبدأ في **أعلى طابور** (أعلى أولوية)

القاعدة 2: إذا استهلكت كل Time Quantum:

- **تنزل** لطابور أدنى (أولوية أقل)
- **السبب:** يبدو أنها CPU-bound

القاعدة 3: إذا تركت CPU قبل انتهاء Quantum:

- **تبقى أو ترتفع** في الأولوية
- **السبب:** يبدو أنها I/O-bound

القاعدة 4: Aging:

- إذا انتظرت طويلاً ← **ترتفع** للطابور الأعلى
- **يمنع Starvation** ✅

📊 المثال النموذجي

```text
Queue 0 (Highest): RR with q=8 ms
Queue 1 (Medium):  RR with q=16 ms
Queue 2 (Lowest):  FCFS

العملية الجديدة → Queue 0
إذا لم تنته في 8 ms → Queue 1
إذا لم تنته في 16 ms → Queue 2
```

✅ المميزات

1. يفصل العمليات حسب السلوك:

- CPU-bound → تنزل تدريجياً
- I/O-bound → تبقى عالية

2. يمنع Starvation:

- بفضل **Aging**

3. مرونة كاملة:

- يمكن تعديل كل شي

⚙️ معاملات MLFQ

يتحدد بـ:

1. **عدد الطوابير**
2. **خوارزمية كل طابور**
3. **شروط الترقية** (متى ترتفع)
4. **شروط الخفض** (متى تنزل)
5. **أي طابور تبدأ فيه العملية**

⭐ **للامتحان:**

- السؤال: "ما الفرق بين MLQ و MLFQ؟"
- الجواب: **MLFQ تسمح بالانتقال بين الطوابير**

📊 جدول المقارنة الشامل

الخوارزمية	النوع	Waiting Time	المميزة	العيب الرئيسي
FCFS	Non-Pre	**طويل جداً**	بسيط	**Convoy Effect**
SJF	كلاهما	**Optimal (أقل)**	الأفضل رياضياً	**صعب التنفيذ**
Priority	كلاهما	متوسط	مرن	**Starvation**
RR	Preemptive	أطول من SJF	**Response Time ممتاز**	Context Switch كثير
MLQ	كلاهما	يعتمد	تنظيم جيد	**Starvation** للدنيا
MLFQ	كلاهما	متوازن	**الأذكى**	**الأعقد**

المعايير (اختصار: CUTRW)

```text
C: CPU Utilization → Maximize
U: (Thro)Ughput → Maximize
T: Turnaround Time → Minimize
R: Response Time → Minimize
W: Waiting Time → Minimize ⭐ (الأهم)
```

الخوارزميات

```text
FCFS: بسيط - Convoy Effect ❌
SJF: Optimal ✅ - صعب التنفيذ ❌
Priority: مرن - Starvation ❌ (حل: Aging)
RR: Response Time ✅ - Context Switch كثير
MLQ: منظم - لا انتقال
MLFQ: ذكي ✅ - معقد
```

المشاكل والحلول

```text
Convoy Effect → SJF بدلاً من FCFS
Starvation → Aging
Response Time بطيء → RR
لا نعرف Burst → Exponential Average
```

جدولة الانضمة متعددة المعالجات

🎯 الصورة الكاملة

التحدي الجديد:

```text
معالج واحد: بسيط - اختر عملية وشغلها ✅
عدة معالجات: معقد - كيف نوزع الشغل؟ 🤔
```

1️⃣ القسم الأول: AMP vs SMP

🔑 نهجان أساسيان

A. Asymmetric Multiprocessing (AMP)

المبدأ: "معالج واحد هو الرئيس"

الهيكل:

```text
Master Processor (الرئيس):
  - يتخذ كل قرارات الجدولة
  - يدير نظام التشغيل
  - يوزع المهام
  ↓
Slave Processors (العمال):
  - ينفذون كود المستخدم فقط
  - لا يتخذون قرارات
```

المميزات:

- ✅ **بسيط جداً** في التصميم
- ✅ **سهل البرمجة**
- ✅ **لا تعقيدات تزامن**

العيوب:

- ❌ **Bottleneck:** Master هو عنق الزجاجة
- ❌ **Single Point of Failure:** إذا Master فشل = انهيار النظام
- ❌ **سوء استغلال الموارد**

⭐ **للحفظ:** AMP = **معالج رئيسي** يقرر - الباقي ينفذون

B. Symmetric Multiprocessing (SMP)

المبدأ: "كل المعالجات متساوية"

الهيكل:

```text
كل المعالجات:
  - متساوية في القدرات [web:154]
  - تجدول نفسها بنفسها (Self-scheduling)
  - تشارك نفس الموارد [web:154]
  - تشارك نفس الذاكرة [web:154]
```

الخصائص:

- ✅ **Identical:** كل المعالجات متطابقة
- ✅ **Shared Memory:** التواصل عبر ذاكرة مشتركة
- ✅ **Equal Access:** وصول متساوٍ للموارد

المميزات:

- ✅ **استغلال أفضل للموارد**
- ✅ **Scalability:** سهل إضافة معالجات
- ✅ **Fault Tolerance:** فشل معالج ≠ انهيار النظام
- ✅ **أداء أفضل**

العيوب:

- ❌ **أكثر تعقيداً** في التصميم
- ❌ **يحتاج تزامن معقد**
- ❌ **أغلى**

📊 جدول المقارنة

الميزة	AMP	SMP
التعقيد	بسيط	**معقد**
القرارات	Master فقط	**كل المعالجات**
استغلال الموارد	ضعيف	**ممتاز**
Bottleneck	✅ موجود	**لا يوجد**
Fault Tolerance	ضعيف	**قوي**
التكلفة	أرخص	**أغلى**
الاستخدام الحديث	نادر	**معيار**

🌍 الأنظمة الحديثة

كل الأنظمة الحديثة تستخدم SMP:

- Windows ✅
- Linux ✅
- macOS ✅
- Android ✅

⭐ **للامتحان:**

- السؤال: "ما النهج المعياري في الأنظمة الحديثة؟"
- الجواب: **SMP - Symmetric Multiprocessing**

2️⃣ القسم الثاني: تنظيم الطوابير

🔑 في SMP، أين نضع Ready Queue؟

خياران:

A. Common Ready Queue (طابور مشترك)

المبدأ: "طابور واحد كبير للكل"

الهيكل:

```text
Ready Queue (مشترك)
[P1, P2, P3, P4, P5]
↓  ↓  ↓
CPU1 CPU2 CPU3
```

الآلية:

- كل معالج **يسحب** من نفس الطابور

المميزات:

- ✅ **توازن تلقائي** - لا معالج فاضي
- ✅ **بسيط نظرياً**

العيوب الخطيرة:

- ❌ **يحتاج Locks** لمنع تعارض الوصول
- ❌ **Contention عالي** - كل المعالجات تتنافس
- ❌ **أداء ضعيف** بسبب الانتظار على Lock
- ❌ **Scalability سيئة** - كلما زادت المعالجات زاد التعارض

⭐ **للحفظ:** Common Queue = **بسيط** لكن **Contention عالي**

B. Per-Core Run Queues (طابور لكل معالج)

المبدأ: "كل معالج له طابوره الخاص"

الهيكل:

```text
CPU1 → Queue1: [P1, P4]
CPU2 → Queue2: [P2, P5]
CPU3 → Queue3: [P3, P6]
```

الآلية:

- كل معالج **يسحب من طابوره الخاص**
- **لا تعارض** على الطابور

المميزات:

- ✅ **لا Locks مطلوبة**
- ✅ **أسرع بكثير**
- ✅ **Cache Efficiency أفضل** (سنشرحها لاحقاً)
- ✅ **Scalability ممتازة**
- ✅ **الأكثر شيوعاً** في الأنظمة الحديثة

العيوب:

- ❌ **قد يحدث عدم توازن** - معالج مشغول وآخر فاضي
- ❌ **يحتاج Load Balancing** (سنشرحه لاحقاً)

⭐ **للامتحان:**

- السؤال: "ما التنظيم الأكثر شيوعاً؟"
- الجواب: **Per-core Queues**

3️⃣ القسم الثالث: Multicore & Multithreading

🔑 المعالجات الحديثة

الواقع:

- معالج واحد به **عدة أنوية** (Cores)
- النواة الواحدة قد تشغل **عدة خيوط** (Hardware Threads)

⚠️ مشكلة Memory Stall

المشكلة:

```text
المعالج سريع جداً: GHz (مليارات العمليات/ثانية)
الذاكرة بطيئة: 100-200 ns للوصول

النتيجة: المعالج ينتظر البيانات ❌
```

النسبة:

- المعالج **أسرع بـ 100-1000 مرة** من الذاكرة
- المعالج يقضي **30-50% من وقته ينتظر!**

التشبيه:

- عامل سريع جداً
- لكن الأدوات تصله ببطء
- **يقف فاضياً** معظم الوقت ❌

✅ الحل: Hardware Multithreading

الفكرة الذكية:

```text
Thread 1 ينتظر البيانات من الذاكرة
↓
بدلاً من الوقوف:
↓
النواة تنتقل لـ Thread 2 وتشغله ✅
↓
عندما بيانات Thread 1 تصل:
↓
تعود له
```

الفائدة:

- ✅ **النواة مشغولة دائماً**
- ✅ **لا وقت ضائع**
- ✅ **استغلال أفضل** بكثير

أمثلة:

- **Intel Hyper-Threading:** نواة واحدة = 2 خيط منطقي
- **IBM POWER8:** نواة واحدة = 8 خيوط

🔄 مستويان للجدولة

عند وجود Hardware Threads:

المستوى 1: Software Thread → Logical CPU

- نظام التشغيل **يجدول Software Threads**
- على **المعالجات المنطقية** (Logical CPUs)
- **نظام التشغيل لا يعرف** أنها hardware threads

المستوى 2: Hardware Thread → Physical Core

- **المعالج نفسه** يجدول Hardware Threads
- على **النواة الفعلية**
- **تلقائياً بالـ Hardware**

التشبيه:

```text
نظام التشغيل يظن: 8 معالجات
الواقع: 4 أنوية × 2 = 8 hardware threads منطقية
```

⭐ **للحفظ:** Hardware Multithreading = **حل Memory Stall** - النواة تنتقل لخيط آخر

4️⃣ القسم الرابع: Load Balancing

🔑 المشكلة

عند استخدام Per-core Queues:

```text
CPU1: [P1, P2, P3, P4, P5] ← مشغول جداً ❌
CPU2: [P6] ← شغل قليل
CPU3: [] ← فاضي تماماً! ❌
```

النتيجة:

- ❌ **سوء استغلال** - CPU فاضي بينما آخر مشغول
- ❌ **Waiting Time عالي** لبعض العمليات

✅ الحل: Load Balancing

الهدف: توزيع الشغل بالتساوي على كل المعالجات

نوعان:

A. Push Migration

المبدأ: "أدفع الشغل للفاضي"

الآلية:

```text
عملية دورية تراقب الأحمال:
↓
إذا وجدت CPU مزدحم:
↓
تدفع (Push) عمليات → CPU فاضي
```

الخصائص:

- ✅ **Proactive** (استباقي)
- ✅ يمنع الازدحام قبل حدوثه
- ❌ يحتاج **مراقبة مستمرة**

B. Pull Migration

المبدأ: "الفاضي يسحب شغل"

الآلية:

```text
CPU فاضي:
↓
يبحث عن CPU مشغول:
↓
يسحب (Pull) عمليات منه
```

الخصائص:

- ✅ **Reactive** (تفاعلي)
- ✅ **لا مراقبة مستمرة**
- ❌ يحدث **بعد** حدوث الفراغ

🐧 Linux: الاثنان معاً

Linux يستخدم:

- ✅ **Push Migration** (دورياً)
- ✅ **Pull Migration** (عند الفراغ)
- **الأفضل من العالمين** ✅

⭐ **للامتحان:**

- السؤال: "ما الفرق بين Push و Pull Migration؟"
- الجواب: **Push = دفع للفاضي | Pull = الفاضي يسحب**

5️⃣ القسم الخامس: Processor Affinity

🔑 المفهوم الأساسي

الملاحظة المهمة:

```text
عملية تعمل على CPU1:
↓
بياناتها تُحمل في Cache الخاص بـ CPU1
↓
"Warm Cache" ✅ - سريع جداً
```

ماذا لو نقلناها لـ CPU2؟

```text
Cache الخاص بـ CPU2 فارغ من بياناتها
↓
"Cold Cache" ❌
↓
يجب إعادة تحميل كل البيانات من الذاكرة
↓
بطء شديد! ❌
```

📊 تأثير Cache

الأرقام:

- **Cache hit:** 1-10 ns (سريع جداً ✅)
- **Cache miss:** 100-200 ns (بطيء جداً ❌)
- **الفرق: 10-100 مرة!**

الفكرة:

- إبقاء العملية على **نفس المعالج** = **Cache دافئ** = **أداء أفضل**

⭐ **التعريف:**

Processor Affinity: ميل العملية للبقاء على نفس المعالج

📊 نوعان من **Affinity**

A. Soft Affinity (ناعمة)

المبدأ: "النظام يحاول بس ما يوعد"

الخصائص:

- النظام **يحاول** إبقاء العملية على نفس CPU
- **لكن قد ينقلها** إذا احتاج للـ Load Balancing
- **لا ضمانات**

متى ينقل؟

- CPU الحالي **مزدحم جداً**
- CPUs أخرى **فاضية**
- **الأولوية** للتوازن

⭐ **للحفظ:** Soft Affinity = **يحاول** - لكن قد ينقل

B. Hard Affinity (صارمة)

المبدأ: "العملية تحدد - النظام ملزوم"

الخصائص:

- العملية/المبرمج **يحدد CPUs محددة**
- النظام **لا يمكنه** نقلها لغيرها
- **ضمان 100%**

الاستخدام:

- ✅ تطبيقات **Real-time**
- ✅ تطبيقات **حرجة الأداء**
- ✅ عند **NUMA** (سنشرحها)

⭐ **للحفظ:** Hard Affinity = **إلزامية** - لا يمكن النقل

⚖️ المقايضة (Trade-off)

المشكلة:

```text
إذا ثبّتنا العملية على CPU معين:
↓
هذا الـ CPU قد يصبح مزدحماً
↓
بينما CPUs أخرى فاضية
↓
سوء استغلال! ❌
```

التوازن:

- **Soft Affinity:** توازن بين Cache و Load Balance ✅
- **Hard Affinity:** Cache ممتاز لكن قد يسبب عدم توازن

NUMA (Non-Uniform Memory Access) 🌐

🔑 المشكلة في الأنظمة الكبيرة

الهيكل:

```text
:Socket 1:                Socket 2
CPU1, CPU2              CPU3, CPU4
Memory Bank 1           Memory Bank 2
```

الواقع القاسي:

- **Local Memory:** CPU1 → Memory Bank 1 = **سريع** (100 ns) ✅
- **Remote Memory:** CPU1 → Memory Bank 2 = **بطيء** (200-300 ns) ❌
- **الفرق: 2-3 مرات!**

⚠️ الخطر

السيناريو السيء:

```text
عملية بياناتها في Memory Bank 1
↓
Load Balancing ينقلها لـ CPU3 (Socket 2)
↓
الآن كل وصول للذاكرة = Remote Access
↓
الأداء ينهار 30-50%! ❌ [web:157]
```

✅ الحل: NUMA-Aware Scheduling

المبدأ:

- **لا تنقل** عملية لـ Socket بعيد عن ذاكرتها
- أو **انقل الذاكرة** مع العملية

Linux NUMA Balancing:

- يراقب **Memory Access Patterns**
- ينقل **Pages** للـ Socket الصحيح
- أو ينقل **Task** للـ Socket الصحيح

⭐ **للامتحان:**

- السؤال: "ما خطر NUMA؟"
- الجواب: **Remote Memory Access أبطأ 2-3 مرات**

6️⃣ القسم السادس: Heterogeneous Multiprocessing (HMP)

🔑 المفهوم الجديد

الفكرة: "ليس كل المعالجات متساوية!"

أين ظهر؟

- **الهواتف الذكية** والأجهزة المحمولة
- ARM big.LITTLE

📱 ARM big.LITTLE

الهيكل:

```text
big Cores (الكبيرة):
  - سريعة جداً ✅
  - أداء عالي ✅
  - تستهلك طاقة عالية ❌
  - للمهام الثقيلة

LITTLE Cores (الصغيرة):
  - أبطأ نسبياً
  - أداء متوسط
  - موفرة للطاقة جداً ✅
  - للمهام الخفيفة
```

المثال:

- 4 big cores + 4 LITTLE cores = 8 أنوية إجمالاً

🎯 هدف HMP

الهدف الذهبي:

- **توفير البطارية** مع **أداء جيد**

كيف؟

```text
مهمة خفيفة (تصفح النت):
→ LITTLE cores ✅
→ بطارية تدوم طويلاً

مهمة ثقيلة (لعبة):
→ big cores ✅
→ أداء ممتاز
```

النتيجة:

- **توفير حتى 75% من الطاقة** في بعض الحالات

🧠 Global Task Scheduling

الوضع المتقدم:

- استخدام **كل الأنوية** في نفس الوقت

التوزيع الذكي:

```text
Threads عالية الأولوية → big cores
Threads منخفضة الأولوية → LITTLE cores
Background Tasks → LITTLE cores
Foreground Apps → big cores
```

🔧 تحدي الجدولة

المجدول يجب أن يكون ذكياً:

يجب أن يعرف:

1. **نوع المهمة:** خفيفة أم ثقيلة؟
2. **الأولوية:** مهمة أم عادية؟
3. **حالة البطارية:** ممتلئة أم منخفضة؟
4. **حرارة الجهاز:** ساخن أم عادي؟

القرارات:

```text
إذا البطارية منخفضة:
  → استخدم LITTLE فقط

إذا البطارية ممتلئة + مهمة ثقيلة:
  → استخدم big

إذا الجهاز ساخن:
  → قلل استخدام big
```

⭐ للامتحان:

- السؤال: "ما هدف ARM big.LITTLE؟"
- الجواب: **توفير الطاقة مع أداء جيد**

📊 جدول المقارنة الشامل

الموضوع	المفهوم الأساسي	الفائدة	المشكلة
SMP	كل المعالجات متساوية	استغلال أفضل	تعقيد
Per-core Queue	طابور لكل معالج	أسرع	عدم توازن
Load Balancing	توزيع متساوٍ	استغلال كامل	Overhead
Processor Affinity	البقاء على نفس CPU	Cache دافئ	عدم مرونة
NUMA	ذاكرة قريبة/بعيدة	وعي بالموقع	تعقيد
HMP	أنوية مختلفة	**توفير طاقة**	جدولة معقدة

AMP vs SMP

```text
AMP:
  - Master يقرر
  - Bottleneck ❌
  - بسيط

SMP: ✅ الحديثة
  - كل المعالجات متساوية
  - Self-scheduling
  - أفضل أداء
```

الطوابير

```text
:Common Queue
  - طابور واحد
  - Contention عالي ❌

:Per-core Queue ✅ الشائع
  - طابور لكل CPU
  - أسرع - لا Locks
  - يحتاج Load Balancing
```

Load Balancing

```text
Push: دفع للفاضي
Pull: الفاضي يسحب
Linux: الاثنان معاً ✅
```

Affinity

```text
Soft: يحاول - قد ينقل
Hard: إلزامية

الفائدة: Warm Cache
NUMA: Remote Memory بطيء 2-3x
```

HMP

```text
:big.LITTLE
  - big: سريع + طاقة عالية
  - LITTLE: بطيء + موفر
  - الهدف: توفير 75% طاقة
```

الجدولة في الوقت الفعلي وامثلة على نضم التشغيل

📍 الجزء الأول: Real-Time Scheduling

1️⃣ القسم الأول: Soft vs Hard Real-Time

🔑 ما هي أنظمة الوقت الفعلي؟

التعريف:

- أنظمة يجب أن **تستجيب** في **وقت محدد**
- **الوقت جزء من الصحة** (Correctness)
- ليس فقط "ماذا" بل **"متى"** أيضاً

المثال:

```text
نظام عادي: احسب 1+1 = 2 ✅
نظام وقت فعلي: احسب 1+1 = 2 خلال 10 ms ⏱️
```

📊 النوعان الأساسيان

A. Soft Real-Time (لينة)

التعريف:

- **لا ضمان صارم** للمواعيد النهائية
- **تفويت Deadline = ليس كارثة**
- العمليات الحرجة تأخذ **أولوية عالية فقط**

الخصائص:

- ✅ أولوية للعمليات الحرجة
- ✅ يمكن إعادة الجدولة
- ✅ التأخير **مقبول** لكن غير مرغوب

الأمثلة:

- 📱 **تشغيل فيديو:** إذا تأخر إطار = تقطيع لكن ليس كارثة
- 🎮 **الألعاب:** تأخير طفيف = lag لكن اللعبة تستمر
- 🎵 **الصوت:** بعض التأخير مقبول

النتيجة عند الفشل:

- ❌ **تدهور الأداء** (Performance degradation)
- ❌ **ليس فشل النظام**

⭐ **للحفظ:** Soft Real-Time = **أولوية عالية** - تفويت Deadline **مقبول**

B. Hard Real-Time (صلبة)

التعريف:

- **ضمان صارم** للمواعيد النهائية
- **تفويت Deadline = فشل النظام بالكامل**
- **يجب** الانتهاء قبل الموعد

الخصائص:

- ⚠️ **صارمة جداً**
- ⚠️ **لا مجال للخطأ**
- ⚠️ الوقت **جزء حرج** من الصحة

الأمثلة:

- 🚗 **نظام وسائد الهواء:** يجب النفخ خلال 10 ms وإلا = موت
- ✈️ **أنظمة الطيران:** التحكم يجب أن يكون فورياً
- 🏥 **أجهزة طبية:** جهاز القلب يجب أن يستجيب فوراً
- 🏭 **أنظمة صناعية:** روبوتات التصنيع

النتيجة عند الفشل:

- ❌ **فشل كارثي** (Catastrophic failure)
- ❌ **خطر على الحياة**
- ❌ **خسائر اقتصادية ضخمة**

⭐ **للحفظ:** Hard Real-Time = **ضمان صارم** - تفويت = **فشل كارثي**

📊 جدول المقارنة

الميزة	Soft Real-Time	Hard Real-Time
الضمان	لا يوجد	**صارم**
تفويت Deadline	مقبول	**كارثي**
الأولوية	عالية فقط	**مضمونة**
الأمثلة	فيديو، صوت	وسائد هواء، طيران
النتيجة عند الفشل	تدهور أداء	**فشل نظام**
التعقيد	أقل	**أعلى**

⭐ **للامتحان:**

- السؤال: "ما الفرق الأساسي بين Soft و Hard؟"
- الجواب: **Hard = تفويت Deadline = فشل كارثي**

2️⃣ القسم الثاني: Latency (زمن التأخير)

🔑 أهمية Latency

القاعدة الذهبية:

- أنظمة الوقت الفعلي تحتاج **استجابة فورية**
- **كل millisecond مهم**
- يجب **تقليل Latency** لأدنى حد

📊 نوعان من Latency

A. Interrupt Latency (تأخير المقاطعة)

التعريف:

- الوقت من **وصول Interrupt** حتى **بداية تنفيذ** ISR (Interrupt Service Routine)

الخطوات:

```text
1. Interrupt يصل
   ↓
2. المعالج يكمل التعليمة الحالية
   ↓
3. يحدد نوع Interrupt
   ↓
4. يحفظ الحالة (Context)
   ↓
5. يقفز لـ ISR ← هنا ينتهي Latency
```

ما يؤثر عليه:

- ⏱️ **وقت إكمال التعليمة الحالية**
- ⏱️ **وقت حفظ الحالة**
- ⏱️ **Interrupt Priority**

⭐ **للحفظ:** Interrupt Latency = وصول Interrupt → بداية ISR

B. Dispatch Latency (تأخير التوزيع)

التعريف:

- الوقت لـ **إيقاف عملية وبدء عملية أخرى عالية الأولوية**

الخطوات (مرحلتان):

المرحلة 1: Conflict Phase (مرحلة التعارض)

```text
- إيقاف العمليات في Kernel mode
- تحرير الموارد من عمليات منخفضة الأولوية
- حل التعارضات
```

المرحلة 2: Dispatch Phase (مرحلة التوزيع)

```text
- Context Switch
- جدولة العملية عالية الأولوية
- بداية التنفيذ
```

الرسم:

```text
Conflict Phase    Dispatch Phase
        ↓                 ↓
[=============][==============]
  Context Switch      التعارضات
```

⭐ **للحفظ:** Dispatch Latency = **مرحلتان:** Conflict + Dispatch

✅ كيف نقلل Latency؟

الحل الأساسي: Preemptive Kernels

لماذا؟

```text
Nonpreemptive Kernel:
- عملية في Kernel mode لا يمكن مقاطعتها
- قد تستمر طويلاً
- Dispatch Latency عالي ❌

Preemptive Kernel:
- يمكن مقاطعة حتى Kernel code
- استجابة فورية
- Dispatch Latency منخفض ✅
```

⭐ **للامتحان:**

- السؤال: "كيف نقلل Dispatch Latency في Real-Time؟"
- الجواب: **استخدام Preemptive Kernel**

3️⃣ القسم الثالث: Real-Time Scheduling Algorithms

🔑 خصائص العمليات الدورية

العمليات في Real-Time عادة Periodic (دورية):

المعاملات الثلاثة:

- **p (Period):** الفترة الزمنية بين كل طلب
- **t (Execution Time):** وقت المعالجة الفعلي
- **d (Deadline):** الموعد النهائي

المثال:

```text
عملية تشغيل فيديو:
- p = 33 ms (30 إطار/ثانية)
- t = 5 ms (وقت معالجة الإطار)
- d = 33 ms (يجب الانتهاء قبل الإطار التالي)
```

A. Rate-Monotonic Scheduling (RMS)

🔑 المبدأ

الفكرة:

- **أولويات ثابتة** (Static Priority)
- **الدورة الأقصر = الأولوية الأعلى**
- **ببساطة:** "اللي يتكرر أكثر، أهم"

القاعدة:

```text
إذا p1 < p2
إذن Priority(P1) > Priority(P2)
```

📊 المثال

```text
P1: p=50 ms, t=20 ms
P2: p=100 ms, t=35 ms

RMS يعطي:
P1 أولوية أعلى (لأن 50 < 100)
```

الجدولة:

```text
P1 ← Time: 0-20 يعمل (20 ms)
P2 ← Time: 20-50 يعمل (30 ms)
P1 ← Time: 50-70 يعمل (20 ms) ← دورة جديدة
P2 ← Time: 70-75 يكمل (5 ms متبقية)
...
```

✅ المميزات

1. Optimal للأولويات الثابتة:

- إذا كان **أي** نظام أولويات ثابت يمكنه جدولة مجموعة عمليات
- **RMS يمكنه جدولتها**

2. بسيط:

- ✅ سهل التنفيذ
- ✅ Overhead منخفض

3. متوقع:

- ✅ الأولويات **ثابتة** - لا تتغير
- ✅ سلوك **محدد مسبقاً**

❌ العيوب

1. استغلال CPU محدود:

- **لا يمكن استخدام 100%** من المعالج
- **الحد الأعلى ≈ 69.3%** عندما n → ∞

المعادلة:

```text
CPU Utilization ≤ n(2^(1/n) - 1)

n=1: 100%
n=2: 82.8%
n=3: 78%
n→∞: 69.3%
```

2. قد تفشل في الجدولة:

- إذا الحمل > 69.3%، **قد تفوت مواعيد**

⭐ **للحفظ:** RMS = **أولويات ثابتة** - أقصر دورة = أعلى أولوية

B. Earliest-Deadline-First (EDF)

🔑 المبدأ

الفكرة:

- **أولويات ديناميكية** (Dynamic Priority)
- **الموعد الأقرب = الأولوية الأعلى**
- **ببساطة:** "اللي Deadline أقرب، يشتغل أول"

القاعدة:

```text
إذا Deadline(P1) < Deadline(P2)
إذن Priority(P1) > Priority(P2)
```

📊 المثال

```text
P1: p=50 ms, t=25 ms, deadline=50
P2: p=80 ms, t=35 ms, deadline=80

Time=0:
P1 deadline=50, P2 deadline=80
→ P1 أولى (80 > 50)

Time=25: P1 ينتهي
P2 deadline=80
→ P2 يعمل

Time=50: دورة P1 جديدة
P1 deadline=100, P2 deadline=80
→ P2 أولى (100 > 80)
```

✅ المميزات الكبرى

1. Optimal بشكل مثبت:

- **يمكنه جدولة أي مجموعة** يمكن جدولتها
- **أفضل** من RMS رياضياً

2. استغلال 100% من CPU:

- يمكن استخدام **كل قدرة المعالج**
- **لا حد أعلى** مثل RMS

المعادلة:

```text
إذا: Σ(t_i / p_i) ≤ 1
إذن: EDF يمكنه جدولة كل العمليات ✅
```

3. مرن:

- ✅ يتعامل مع **مواعيد نهائية عشوائية**
- ✅ d ≠ p (الموعد قد يختلف عن الدورة)

❌ العيوب

1. Overhead أعلى:

- **الأولويات تتغير** باستمرار
- حساب **أقرب Deadline** يأخذ وقت

2. صعب التنفيذ:

- أعقد من RMS

3. غير متوقع في Overload:

- عند الحمل الزائد، صعب التنبؤ **من سيفشل**
- RMS أفضل هنا

⭐ **للامتحان:**

- السؤال: "أيهما يمكنه استغلال 100% CPU؟"
- الجواب: **EDF**

📊 مقارنة RMS vs EDF

الميزة	RMS	EDF
الأولويات	ثابتة	**ديناميكية**
الأساس	أقصر دورة	**أقرب deadline**
CPU Utilization	69.3%	**100%**
Overhead	منخفض	**أعلى**
Optimality	للثابتة	**مطلقة**
Overload	متوقع	**غير متوقع**
التعقيد	بسيط	**معقد**

4️⃣ القسم الرابع: POSIX Real-Time Scheduling

🔑 معيار POSIX

POSIX: معيار موحد لأنظمة Unix-like

يوفر فئتان للـ Real-Time:

A. SCHED_FIFO

الخصائص:

- **FIFO Queue** لكل مستوى أولوية
- **Nonpreemptive** بين نفس الأولوية
- **Preemptive** لأولوية أعلى
- العملية تعمل **حتى تنتهي** أو تحجب

الآلية:

```text
أولوية 10: [P1, P2] ← FIFO
أولوية 20: [P3]

P3 يعمل (أعلى أولوية)
عندما ينتهي → P1 يعمل (أول واحد في 10)
```

B. SCHED_RR

الخصائص:

- مثل SCHED_FIFO **لكن مع Time Quantum**
- **Round-Robin** بين نفس الأولوية
- **Preemptive** بين المتساويين

الآلية:

```text
RR with quantum ← [P1, P2, P3] :10 أولوية

P1 يعمل quantum → آخر الطابور
P2 يعمل quantum → آخر الطابور
P3 يعمل quantum → آخر الطابور
P1 مرة أخرى...
```

⭐ **الفرق:** SCHED_RR = SCHED_FIFO + **Time Quantum**

📍 الجزء الثاني: Operating System Examples

5️⃣ القسم الخامس: Linux Scheduling

🔑 CFS (Completely Fair Scheduler)

التاريخ:

- قبل 2.6.23: O(1) Scheduler
- **بعد 2.6.23:** CFS (الحالي)

الفلسفة:

- **العدالة المطلقة** (Completely Fair)
- كل عملية تأخذ **نصيبها العادل** من CPU

📊 Scheduling Classes

Linux يقسم العمليات لفئتين:

1. Real-Time Classes (0-99):

- **SCHED_FIFO:** أولوية 1-99
- **SCHED_RR:** أولوية 1-99
- **أعلى أولوية** في النظام

2. Normal Class (100-139):

- **SCHED_NORMAL (CFS)**: أولوية 100-139
- العمليات العادية

الترتيب:

```text
Real-Time (0-99) → ينفذ أولاً دائماً
Normal (100-139) → بعد انتهاء Real-Time
```

🔧 كيف يعمل CFS؟

المفاهيم الأساسية:

---

1. Nice Value

التعريف:

- قيمة تحدد **نصيب العملية** من CPU
- المدى: **-20 إلى +19**
- **أقل nice = أولوية أعلى**

التفسير:

```text
Nice = -20: "أنا مهم جداً!" → أكبر نصيب
Nice = 0: "عادي" → نصيب متوسط
Nice = +19: "أنا لطيف، أنتظر" → أقل نصيب
```

التحويل للأولوية:

```text
Priority = 120 + Nice Value

Nice=-20 → Priority=100 (أعلى في Normal)
Nice=0 → Priority=120
Nice=+19 → Priority=139 (أدنى في Normal)
```

---

2. Virtual Runtime (vruntime)

التعريف:

- **مقياس وقت** العملية على CPU
- **معدّل** حسب الأولوية

الفكرة:

```text
عملية عالية الأولوية:
  - تأخذ وقت أكثر
  - لكن vruntime يزيد ببطء ✅

عملية منخفضة الأولوية:
  - تأخذ وقت أقل
  - لكن vruntime يزيد بسرعة
```

القاعدة الذهبية:

- **CFS دائماً يختار:** العملية بـ **أقل vruntime**

المثال:

```text
P1: vruntime = 100
P2: vruntime = 150
P3: vruntime = 80

CFS يختار: P3 (أقل vruntime) ✅
```

---

3. Red-Black Tree

الهيكل:

- CFS يستخدم **شجرة حمراء-سوداء**
- مرتبة حسب **vruntime**

الفائدة:

```text
O(log n) للبحث والإدراج ✅
الوصول لأقل vruntime = O(1) ✅ (أقصى اليسار)
```

الآلية:

```text
      [P2: vruntime=150]
      /                \
[P3: 80]           [P4: 200]
   \
 [P1: 100]

أقل vruntime = P3 (أقصى اليسار)
```

🔄 خطوات CFS

```text
1. يختار العملية بأقل vruntime (من الشجرة)
2. العملية تعمل فترة
3. vruntime يزيد (حسب nice value)
4. العملية تُعاد للشجرة
5. الدورة تتكرر
```

⭐ **للحفظ:** CFS = **vruntime + Red-Black Tree** - أقل vruntime يعمل

🌐 NUMA Awareness

CFS ذكي:

- يراعي **NUMA Domains**
- **لا ينقل** عملية بعيداً عن ذاكرتها إلا للضرورة
- **يوازن** بين الأداء والتوازن

6️⃣ القسم السادس: Windows Scheduling

🔑 Priority-Based Preemptive

الخصائص:

- **قائم على الأولويات بالكامل**
- **Preemptive** دائماً
- **32 مستوى أولوية** (0-31)

📊 مستويات الأولوية

التقسيم:

Real-Time Class (16-31):

- **أعلى أولوية** في النظام
- للعمليات الحرجة

Variable Class (1-15):

- للعمليات **العادية**
- **يمكن تغيير أولويتها**

System Idle (0):

- **محجوز** لإدارة الذاكرة
- يعمل فقط عند فراغ النظام

🔧 آلية العمل

القاعدة:

- Windows **دائماً** يختار **أعلى أولوية** جاهزة
- إذا تساوت الأولويات → **Round-Robin**

الخطوات:

```text
1. يبحث من 31 → 0
2. يجد أول عملية جاهزة
3. يعطيها Time Quantum
4. إذا انتهى Quantum:
   - الأولوية تنخفض قليلاً (Variable Class فقط)
   - تعود لآخر الطابور
```

⚡ Priority Boost (التعزيز)

متى يحدث؟

- عملية كانت **منتظرة** وصحت
- **خاصة:** I/O completion

السبب:

```text
عملية كانت تنتظر Keyboard input
↓
المستخدم ضغط زر
↓
Windows يرفع أولويتها مؤقتاً ✅
↓
استجابة سريعة = رضا المستخدم ✅
```

التطبيقات:

- ✅ انتظار boost → **Keyboard** عالي
- ✅ انتظار boost → **Disk** متوسط
- ✅ انتظار boost → **Network** منخفض

🖥️ Foreground vs Background

الفكرة:

- البرنامج **الأمامي** (اللي أنت تستخدمه) **أهم**
- برامج **الخلفية** أقل أهمية

الآلية:

```text
Foreground Process:
  - Time Quantum أطول (عادة 3x) ✅
  - استجابة أفضل

Background Process:
  - Time Quantum أقصر
  - لكن لا يتجمد
```

المثال:

```text
Quantum = 30 ms

Foreground: 90 ms (3x)
Background: 30 ms (1x)
```

⭐ للامتحان:

- السؤال: "كيف Windows يحسّن الاستجابة؟"
- الجواب: **Priority Boost + Foreground أطول**

7️⃣ القسم السابع: Solaris Scheduling

🔑 Six Scheduling Classes

Solaris يدعم 6 فئات:

1. **Time-Sharing (TS)** - الافتراضية
2. **Interactive (IA)**
3. **Real-Time (RT)**
4. **System (SYS)**
5. **Fair-Share (FSS)**
6. **Fixed-Priority (FP)**

سنركز على الثلاثة الأساسية:

A. Time-Sharing (TS)

الخصائص:

- **الفئة الافتراضية** للعمليات
- تستخدم **جداول أولويات**

الفلسفة العكسية:

القاعدة المثيرة:

```text
أولوية عالية = Time Quantum قصير ⏱️
أولوية منخفضة = Time Quantum طويل ⏱️⏱️⏱️
```

لماذا؟

العمليات التفاعلية (I/O-bound):

- عادة **قصيرة** → أولوية عالية
- Time Quantum قصير **كافٍ** لها
- **تنهي وتغادر بسرعة** ✅
- استجابة ممتازة

العمليات الحسابية (CPU-bound):

- تحتاج **وقت طويل** → أولوية منخفضة
- Time Quantum طويل
- **عندما يأتي دورها** تأخذ وقتها ✅

المثال:

```text
Priority 59: Quantum = 20 ms (عالية - قصير)
Priority 30: Quantum = 160 ms (متوسطة)
Priority 0: Quantum = 400 ms (منخفضة - طويل)
```

B. Interactive (IA)

الخصائص:

- **مثل TS** بالضبط
- **لكن:** أولوية **أعلى** لـ Windowing applications

الهدف:

- تحسين **استجابة** واجهة المستخدم
- البرامج المرئية **أسرع**

C. Real-Time (RT)

الخصائص:

- **أعلى أولوية** في النظام
- تسبق **كل الفئات** الأخرى
- لا تتغير أولويتها

⭐ **للحفظ:** Solaris TS = **أولوية عالية** = **Quantum قصير**

📊 جدول المقارنة الشامل

النظام	الأساس	الفئات	الميزة الفريدة
Linux (CFS)	vruntime	2 (RT + Normal)	**Red-Black Tree**
Windows	Priority (0-31)	3 (RT + Var + Idle)	**Priority Boost**
Solaris	Classes	6 فئات	**علاقة عكسية** (Priority/Quantum)

Solaris

```text
Classes 6:
  - TS (افتراضية)
  - IA (Interactive)
  - RT (أعلى)

TS علاقة عكسية:
  أولوية عالية = Quantum قصير ⚡
  أولوية منخفضة = Quantum طويل

السبب: تفاعلية أفضل
```

Real-Time

```text
Soft: أولوية عالية - تفويت مقبول
Hard: ضمان صارم - تفويت = كارثة

Latency نوعان:
1. Interrupt: وصول → ISR
2. Dispatch: Conflict + Dispatch

الحل: Preemptive Kernel

RMS:
- أولويات ثابتة
- أقصر دورة = أعلى أولوية
- CPU ≤ 69.3%
- Optimal للثابتة

EDF:
- أولويات ديناميكية
- أقرب deadline = أعلى أولوية
- CPU = 100% ✅
- Optimal مطلقاً ✅

POSIX:
SCHED_FIFO: حتى تنتهي
SCHED_RR: + Time Quantum
```

Linux (CFS)

```text
Classes: RT (0-99) + Normal (100-139)

CFS:
- vruntime (وقت افتراضي)
- أقل vruntime يعمل ✅
- Red-Black Tree
- O(log n)

Nice: -20 (عالي) → +19 (منخفض)
Priority = 120 + Nice

✅ NUMA Aware
```